

Ex 7: Retrieval-Augmented Generation (RAG) using Vector Database and LLM

Learning Objective:

Develop a Python program to integrate a Vector Database with a Large Language Model (LLM) to build a Retrieval-Augmented Generation (RAG) system that answers questions based on external PDF documents such as LinkedIn profiles or research papers.

Steps:

1. Import Required Libraries (OpenAI, Hugging Face Transformers, Vector Database libraries like FAISS/Chroma, PDF processing libraries)
2. Configure OpenAI API key
(This key should be personal. Do not share with anyone).
3. Load PDF Documents
Use LinkedIn profile PDFs or research paper PDFs as the dataset.
4. Extract Text from PDFs
Convert PDF content into raw text using suitable libraries.
5. Split Text into Chunks
Divide extracted text into smaller chunks for efficient retrieval.
6. Generate Embeddings
Convert text chunks into vector embeddings using embedding models.
7. Store Embeddings in Vector Database
Use FAISS or Chroma to store and index embeddings.
8. Initialize the LLM
Load OpenAI or Hugging Face model for generating responses.
9. Create Retrieval Function
Retrieve relevant document chunks from the vector database based on user query.
10. Build RAG Pipeline
Combine retrieved context with user query and pass it to the LLM to generate answers.
11. Provide User Query
Ask questions related to the content of the PDF documents.
12. Generate Responses
Use retrieved information + LLM to produce accurate answers.

13. Display the Output

Present retrieved context and generated answer.

Program Code:

```
# Import Libraries
import os
import faiss
import numpy as np
import google.generativeai as genai
from sentence_transformers import SentenceTransformer
from PyPDF2 import PdfReader
from google.colab import drive
from google.colab import userdata

# Mount Google Drive
drive.mount('/content/drive')

# Provide PDF Path
pdf_path = input("Enter full PDF path from Google Drive: ")

if not os.path.exists(pdf_path):
    raise FileNotFoundError("Invalid path! Please check your file path.")

# Configure Gemini API Key
GEMINI_API_KEY = userdata.get('GEMINI_API_KEY')
genai.configure(api_key=GEMINI_API_KEY)

# Load PDF
reader = PdfReader(pdf_path)
text = ""

for page in reader.pages:
    text += page.extract_text() or ""
```

```
print("PDF loaded successfully!")
```

```
# Split Text into Chunks
```

```
def split_text(text, chunk_size=300, overlap=50):  
    chunks = []  
    for i in range(0, len(text), chunk_size - overlap):  
        chunks.append(text[i:i + chunk_size])  
    return chunks
```

```
chunks = split_text(text)  
print(f"Total chunks: {len(chunks)}")
```

```
# Generate Embeddings
```

```
embedder = SentenceTransformer("all-MiniLM-L6-v2")  
embeddings = embedder.encode(chunks)
```

```
# Store in FAISS
```

```
dimension = embeddings.shape[1]  
index = faiss.IndexFlatL2(dimension)  
index.add(np.array(embeddings))
```

```
# Initialize Gemini Model
```

```
model = genai.GenerativeModel("gemini-2.5-flash-lite")
```

```
# Retrieval Function
```

```
def retrieve(query, k=3):  
    query_vec = embedder.encode([query])  
    distances, indices = index.search(np.array(query_vec), k)  
    return [chunks[i] for i in indices[0]]
```

```
# RAG Pipeline
```

```
def rag_pipeline(query):  
    retrieved_chunks = retrieve(query)
```

```
context = "\n\n".join(retrieved_chunks)
```

```
prompt = f"""
```

```
Answer the question ONLY using the context below.
```

```
Context:
```

```
{context}
```

```
Question:
```

```
{query}
```

```
Answer:
```

```
"""
```

```
response = model.generate_content(prompt)
```

```
return context, response.text
```

```
# User Query
```

```
query = input("Enter your question: ")
```

```
# Generate Response
```

```
context, answer = rag_pipeline(query)
```

```
# Display Output
```

```
print("\n===== Retrieved Context =====\n")
```

```
print(context)
```

```
print("\n===== Generated Answer =====\n")
```

```
print(answer)
```

Output:

```
Enter full PDF path from Google Drive: /content/drive/MyDrive/NIPS-2017-attention-is-all-you-need-Paper.pdf
PDF loaded successfully!
Total chunks: 131
```

```
Enter your question: What is a Transfromer?
```

```
===== Retrieved Context =====
```

```
shown to perform well on simple-language question answering and
language modeling tasks [28].
To the best of our knowledge, however, the Transformer is the first transduction model relying
entirely on self-attention to compute representations of its input and output without using sequence-
aligned R
```

```
Table 3: Variations on the Transformer architecture. Unlisted values are identical to those of the base
model. All metrics are on the English-to-German translation development set, newstest2013. Listed
perplexities are per-wordpiece, according to our byte-pair encoding, and should not be compared to
```

```
input and output without using sequence-
aligned RNNs or convolution. In the following sections, we will describe the Transformer, motivate
self-attention and discuss its advantages over models such as [14, 15] and [8].
```

```
3 Model Architecture
```

```
Most competitive neural sequence transduction models have a
```

```
===== Generated Answer =====
```

```
The Transformer is the first transduction model relying entirely on self-attention to compute representations of its input and output without using sequence-aligned RNNs or convolution.
```

Learning Outcome:

Students will be able to build a Retrieval-Augmented Generation (RAG) system by integrating vector databases with LLMs, enabling accurate question answering over external PDF documents such as LinkedIn profiles or research papers.